資料結構與演算法

資料結構是什麼

- 稍微回想一下，以下兩者有什麼不同。
- 家中的書櫃
- 圖書館裡的館藏
  - 數量
  - 分類
  - ID-識別規格

資料結構是什麼

- 資料結構（data structure）是指資料儲存的方式，稍微思考一下，有經過整理、排序的書櫃，按照特定的方式進行安排是不是可以讓我們更快找到想要的書。
- 是指用來電腦中儲存資料的結構。
- 目的性=>新增 查詢 刪除 (修改)
- 作用不同

常見的資料結構

- 線性結構：陣列、鏈結串列、堆疊、佇列……
  - 整體結構以線性結構為主。
- 非線性結構：樹狀結構、圖形結構
  - 具有階層性質，或是有相鄰性質

陣列(array)

- 陣列，程式設計最早會接觸到的資料結構。
- 由相同類型的元素（element）的集合所組成的資料結構。
- 元素：資料型別，在程式中常見的元素有整數、字元等(基本資料型別)，也有採用參考資料型別等。
- 記錄方式為在一個連續的記憶體區塊上記錄。
- 由於可以藉由計算方式直接取得陣列中的元素，故這種儲存方式易於讀取。

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
12	5	3	7	9	29	11	23	15	41	42				

陣列

- 參考用圖形(實際上在電腦中並非如此，僅作為參考、學習時使用）：

索引值	0	1	2	3	4	5	6	7	8	9
資料值	12	5	3	7	9	11	23	15	41	42

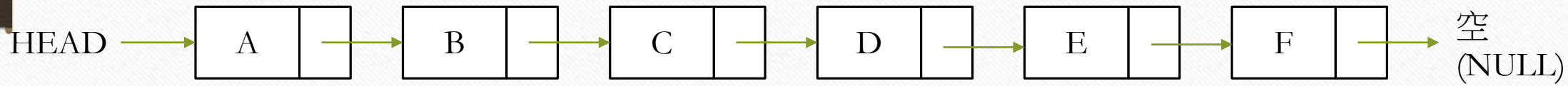
- 這種直接利用索引值，透過計算直接取得資料的方式，稱為隨機存取

陣列

- 早期的資料結構
- 使用前預先請求空間。
- 因為如此使得他會有一些特性。
  - 請求空間以後大小固定，不能再改變
  - 記憶體中有空間連續性的表現，中間不會存在其他程式需要調用的數據
  - 早期語言陣列常不做下界判斷，對於電腦中存在有風險(越界操作)

鏈結串列

- 鏈結串列linked list是一種將前一筆資料與下一筆資料相連接的結構。
- 結構範例

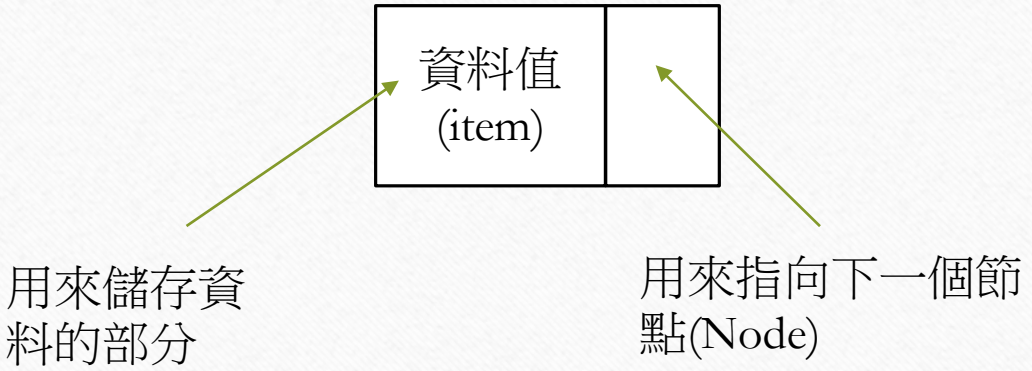


- 陣列找資料的速度較快，但鏈結串列找資料速度較慢
- 陣列要新增刪除資料速度很慢，但鏈結陣列要新增刪除時很快

鏈結串列

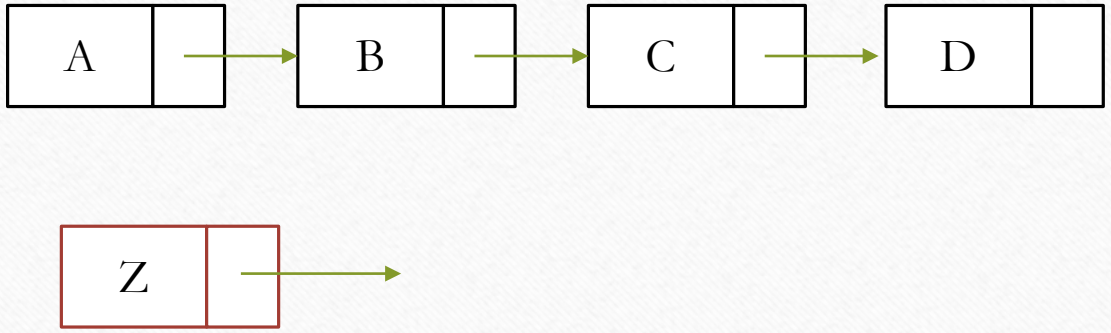
- 結構解說：
- 鏈結串列中的節點分成兩個部分
  一個是儲存資料值
  一個儲存下一個節點的位置

節點(Node)：鏈結串列中儲存資料的一個端點。



鏈結串列的新增與刪除

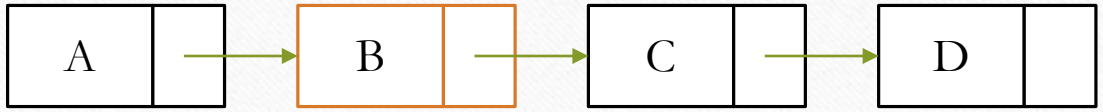
- 鏈結串列的新增



要插入的
資料

鏈結串列的新增與刪除

- 鏈結串列的刪除



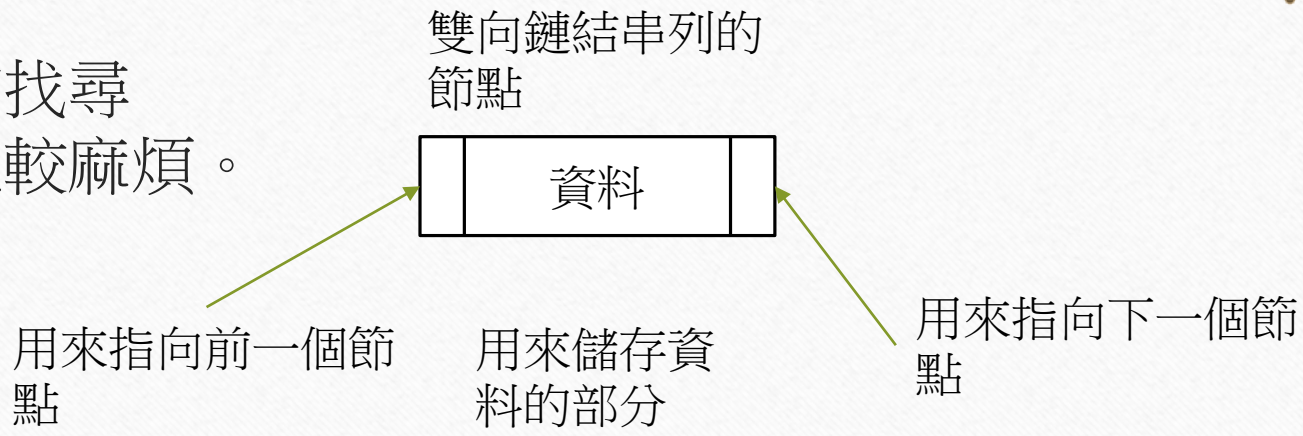
要刪除的
資料

鏈結串列的變化型

- 雙向鏈結串列
- 循環鏈結串列

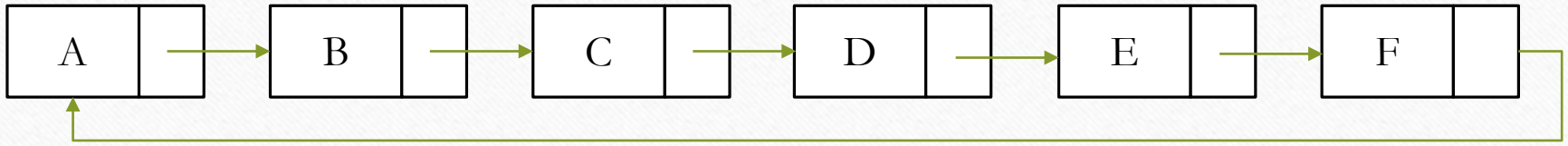
雙向鏈結串列

- 通常應用在需要來回的找尋資料。
- 因為一般的鏈結串列(單向鏈結串列)沒有辦法往前找尋，所以產生了這個資料結構。
- 雙向連結串列雖然可以往前找尋
  但增加、刪除節點會變得比較麻煩。

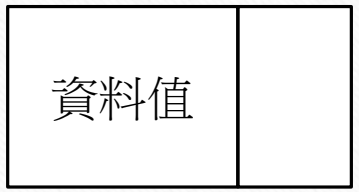


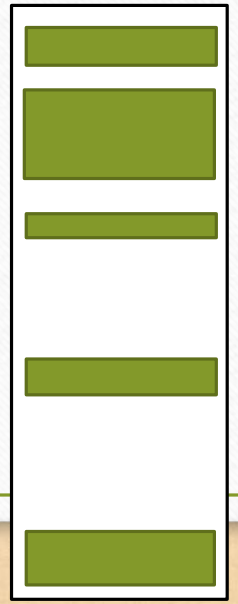
循環鏈結串列

- 循環鏈結串列適合不停重複找尋插入點的工作。
- 從循環鏈結串列中的任何一個節點出發都能找到任何其他節點。



- 節點->占用的空間 不連續的。=>分散在記憶體各處，彈性的資料結構。
- 陣列=>確保有一定的連續空間可以使用
- 鏈結串列=>整體上需要的空間比較大，但是他可以去利用零碎的空間。





明天做什麼

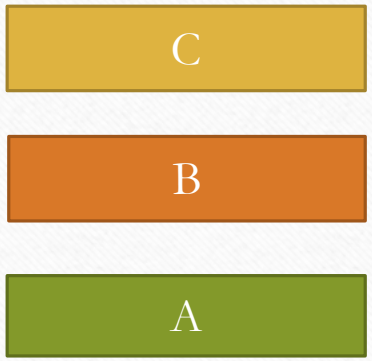
- 完成思科學院的課程
- 找到排序法(有空的同學再做)
- python先修(O)

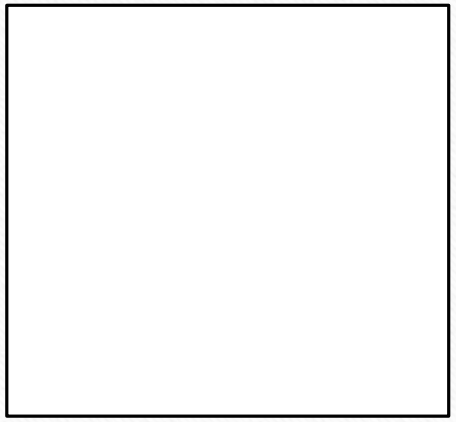
堆疊stack

- 堆疊通常只做兩件事情，推入與彈出。
- 推入，加入資料進入堆疊。(push)
- 彈出，將資料從堆疊彈出。(pop)
- [面試常考題、國家考試]
- 這種資料結構有後進先出的特性。(LIFO, Last-in-First-out)

堆疊

- 參考圖





堆疊

堆疊

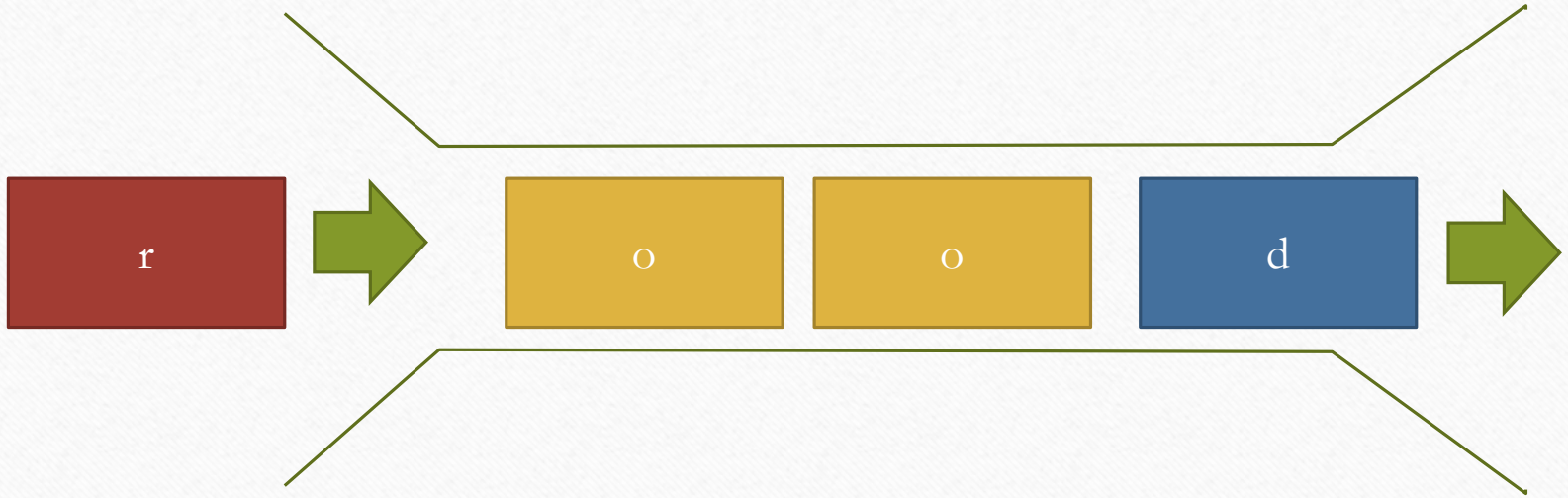
- 堆疊的應用場合：
  瀏覽紀錄
  文書軟體一類的復原
  解決走迷宮問題

佇列

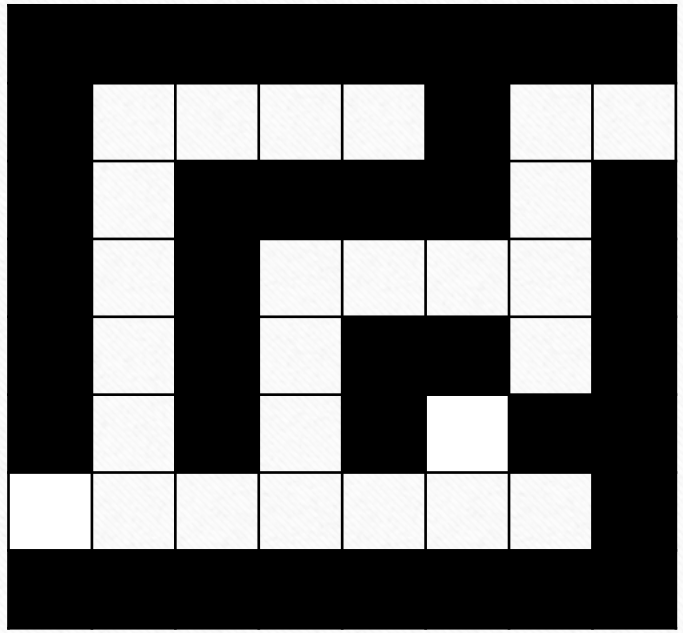
- 佇列(Queue)也只做兩件事
- 加入(enqueue)與取出(dequeue)
- 佇列的加入要從最尾端加入。
- 佇列的取出只能從最前頭。
- 佇列的特色是先進入的先出來(FIFO, First-in-First-out)

佇列

- 參考圖



走迷宮



如何走迷宮
演算法有關係
走迷宮>策略
POP 時機點(V)
時機=>死路
POP->刪除
時機點=>走出迷宮
POP=>儲存的目的

- Object[ ]<=陣列Array
- ArrayList=>Array完成List的工作
- 尺寸： ArrayList:彈性的
  Array:固定的

- 緩衝器：資料處理(3 words/s) 資料的儲存(10 words/s)
- 5=>純資料處理=>處理不來
- 5=>存到緩衝器=>3 words/s 處理 =>處理得來

佇列

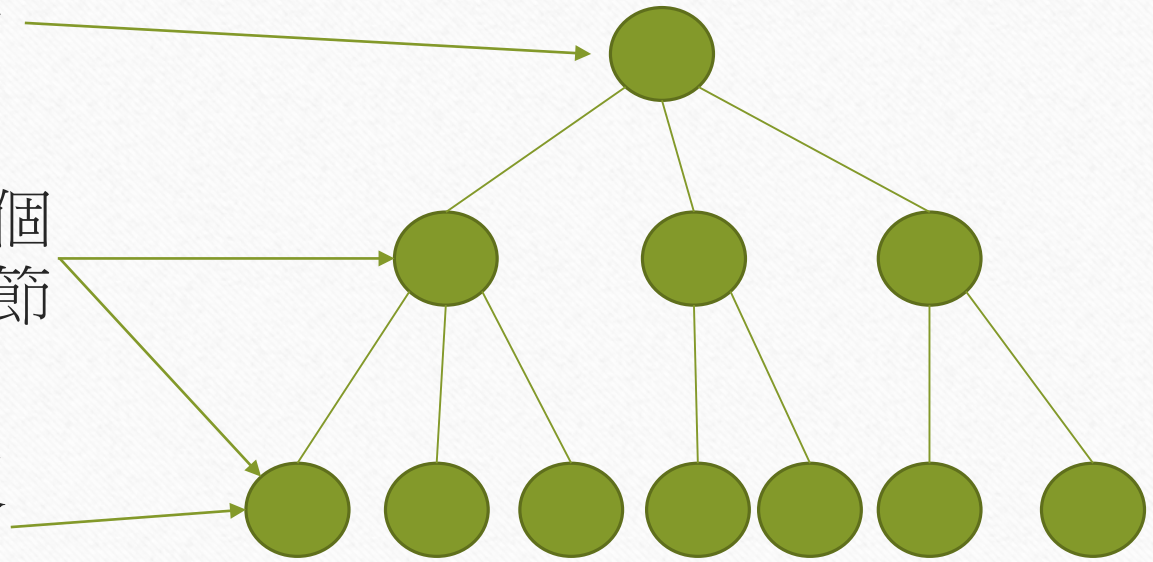
- 佇列的應用場合
  - 排隊
  - 緩衝器

樹(tree)

- 樹是一種有樹狀結構的資料結構。
- 樹的特性如下：
  - 每個節點都只有有限個子節點或無子節點；
  - 沒有父節點的節點稱為根節點；
  - 每一個非根節點有且只有一個父節點；
  - 除了根節點外，每個子節點可以分為多個不相交的子樹
  - 樹裡面的節點不會構成循環

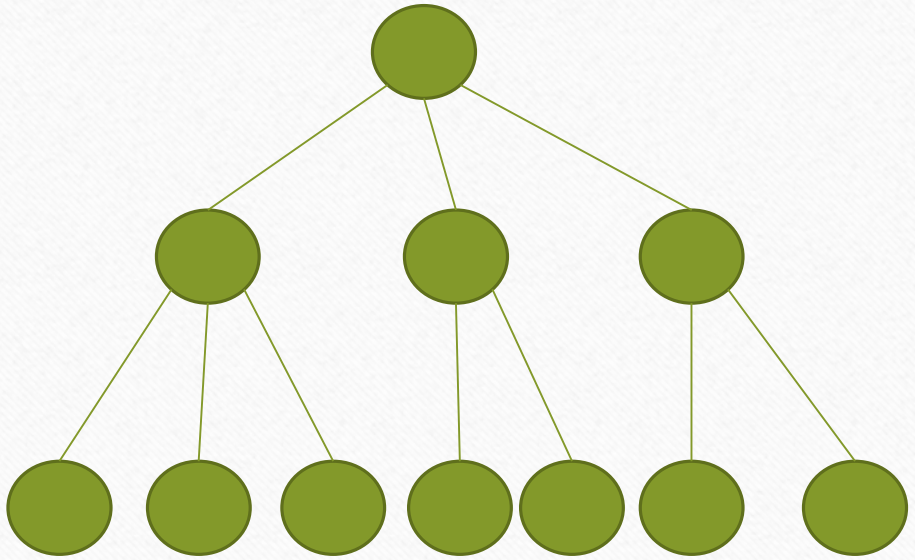
樹

- 樹根(root)用來表示樹的最上方節點。
- 父節點與子節點：用來表示兩個階層的上下關係，上層稱為父節點，下層稱為子節點。
- 樹葉(Leaf)：最末端的節點，沒有子節點的節點。



樹

- 高度：一顆樹的樹根到樹葉的最大距離。=>高度為3
- 階層：該節點處在的位置，樹根為1。
- 分支度：一個節點所具有的子節點數。



樹的作用

- 樹的作用主要結合鏈結串列容易新增刪除的特性，同時也減少搜尋所需的時間。

二元樹

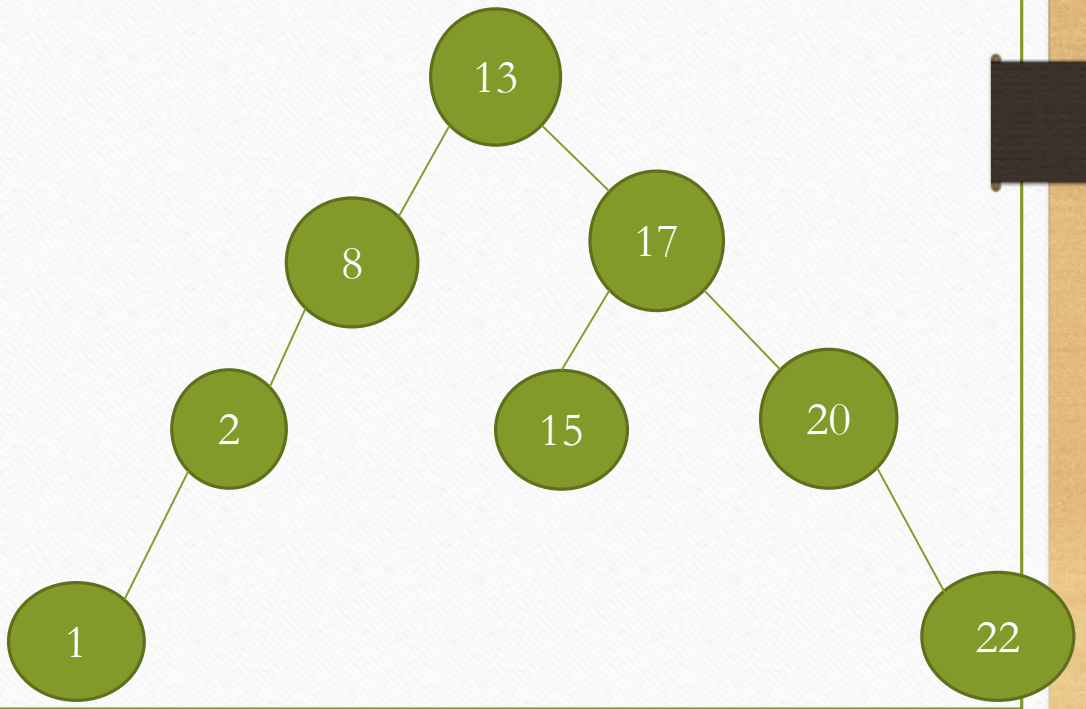
- 二元樹：最大分支度為2的樹0,1,2

二元搜尋樹

- 由於樹本身有很多相關的、不同的種類與特性(這個一講可能會花不少時間，同時在短期內也比較沒那麼實用)，所以簡單介紹其中一種樹。
- 二元搜尋樹，是一個有被定義的二元樹。
  - 定義：他是一個二元樹，每一個節點最多只有兩個子節點。
  - 比父節點小的節點則往左子樹新增
  - 比父節點大的往右子樹新增

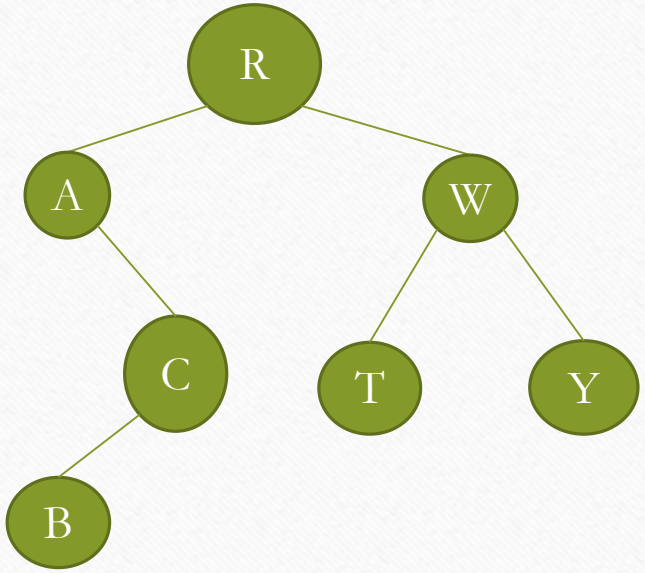
二元搜尋樹

- 二元搜尋樹示範
- 1~8
- 1~4



二元搜尋樹

- 二元搜尋樹示範



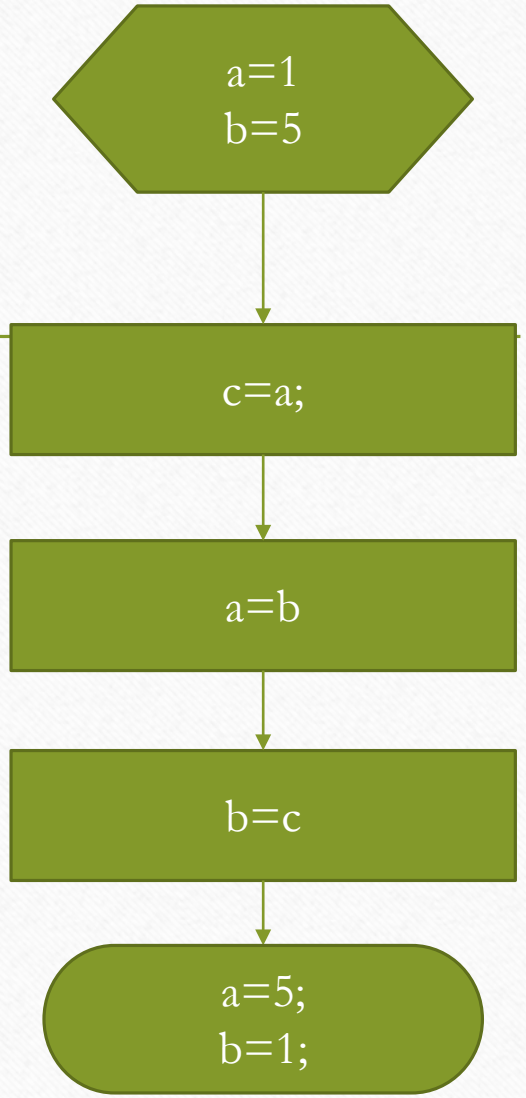
演算法

演算法之前

- 在談論演算法之前，我們先知道一下程式設計常見的結構。
- 循序=>按照順序
- 選擇=>單邊選擇 雙邊選擇 巢狀 多重
- 循環=>前測迴路 後測迴路

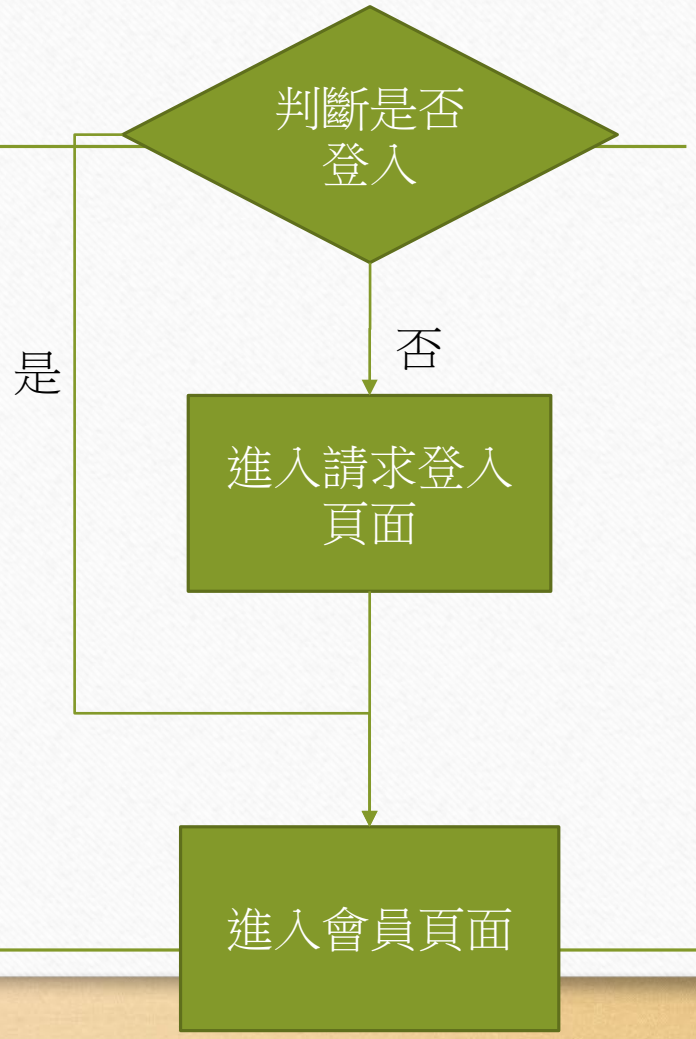
循序結構

- 最常見的結構，也是程式在執行時的方式。
- 執行順序由上往下。
- 使用範例，2個變數交換。

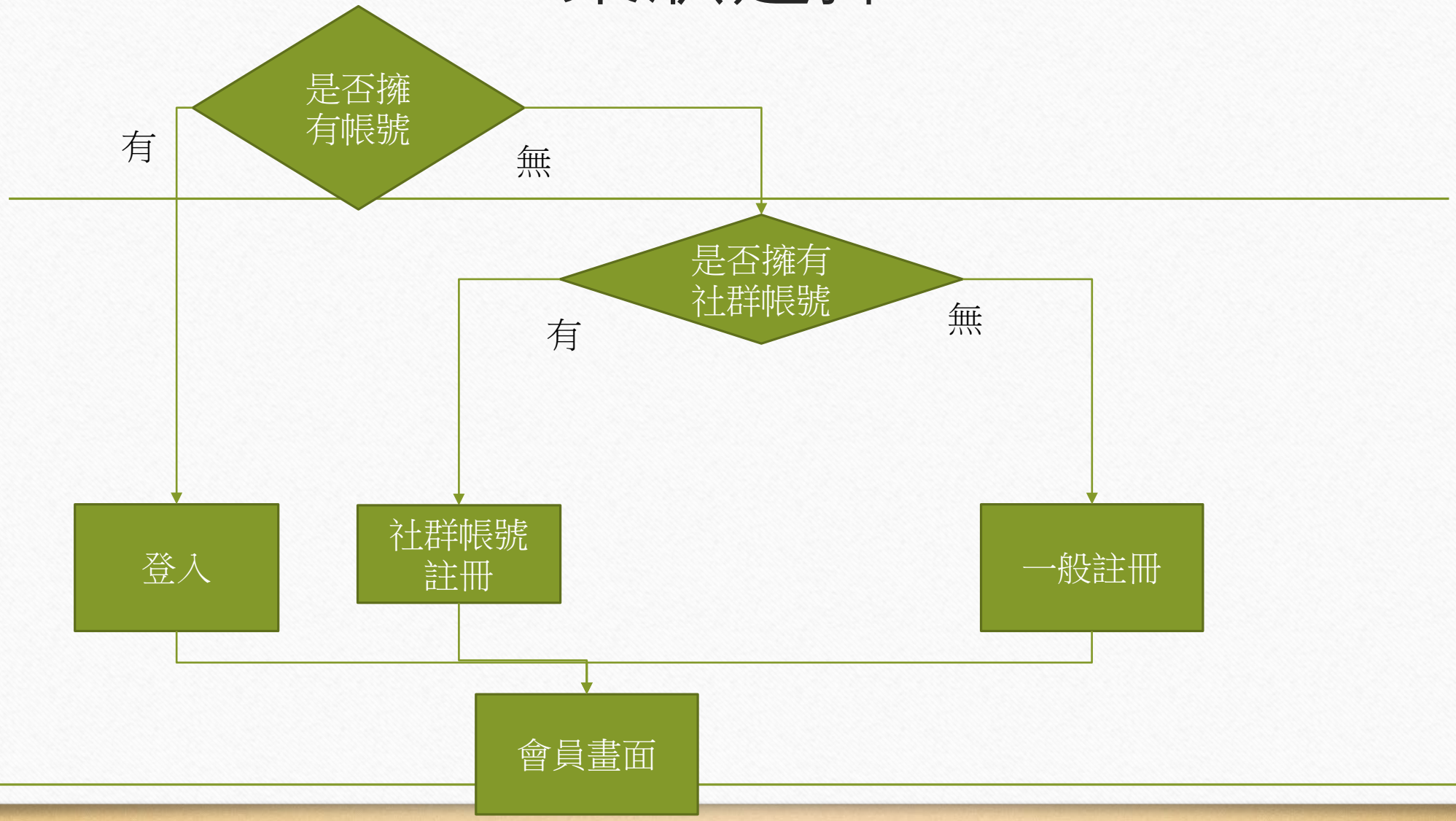


選擇結構

- 用來進行判斷或是選擇的結構。
- 例如：成績輸入，判斷是否登入。



巢狀選擇

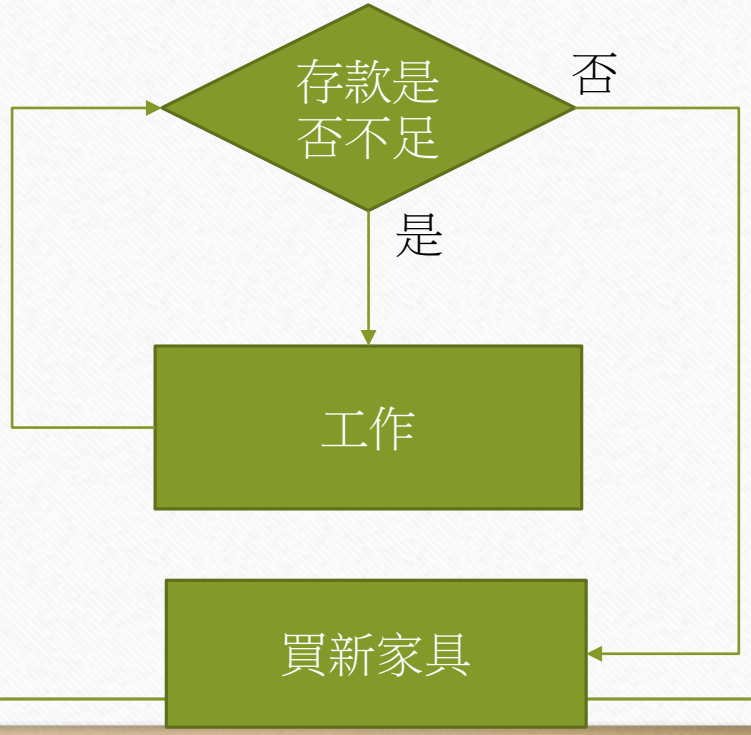


選擇結構

- 常見到的敘述方式：
- 單邊選擇結構：if 條件 then 敘述
- 雙邊選擇結構：if 條件 then 敘述 else 敘述
- 巢狀選擇結構：if 條件 then 條件 then 敘述2 else 敘述3 else if 條件 then 敘述2 else 敘述3
- 多重選擇結構：switch(輸入值) {條件值 敘述..... }

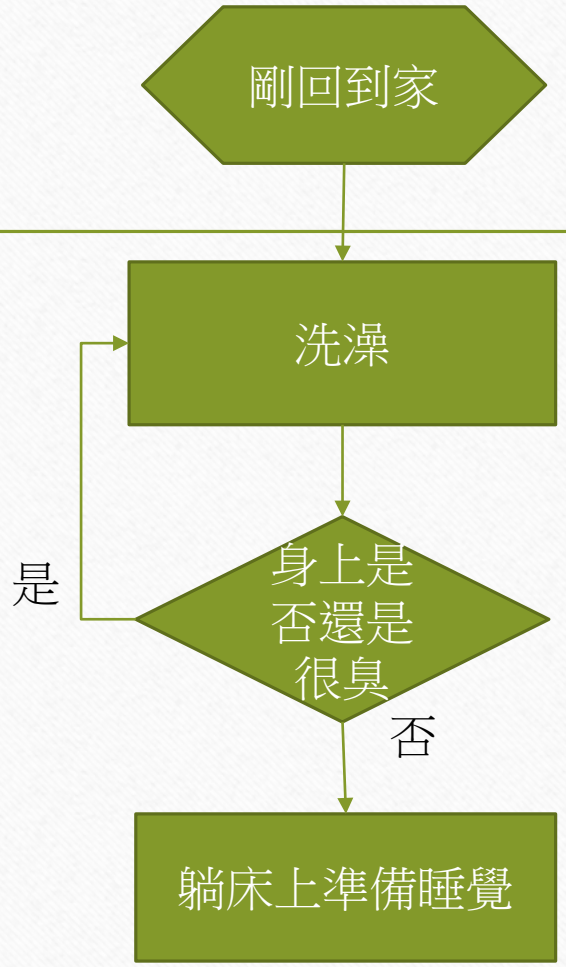
循環結構

- 用來進行一連串的動作，直到完成脫離條件。
- 通常用來讀取資料(疊代)，或是完成有規則可循的動作。



循環結構

- 循環結構分為前測結構與後測結構。
- 其中最大的差異在於會先執行敘述或是優先執行判斷。



OK 可以進入演算法了

演算法說什麼

- 演算法，其實就是解決問題的步驟。
- 演算法基本上具有5個特性(高德納歸納)
  - 輸入
  - 輸出
  - 明確性
  - 有限性
  - 有效性

演算法

- 輸入：一個演算法至少要0個以上的輸入。
- 輸出：一個演算法至少要1個以上的輸出，輸出值是演算法計算的結果。
- 明確性：演算法的結果是精確地符合要求或期望，不能有歧異。
- 有限性：演算法必須在有限個步驟內完成任務。
- 可行性：能夠實現，演算法中描述的操作都是可以通過已經實現的基本運算執行有限次來實現。

五大常用演算策略

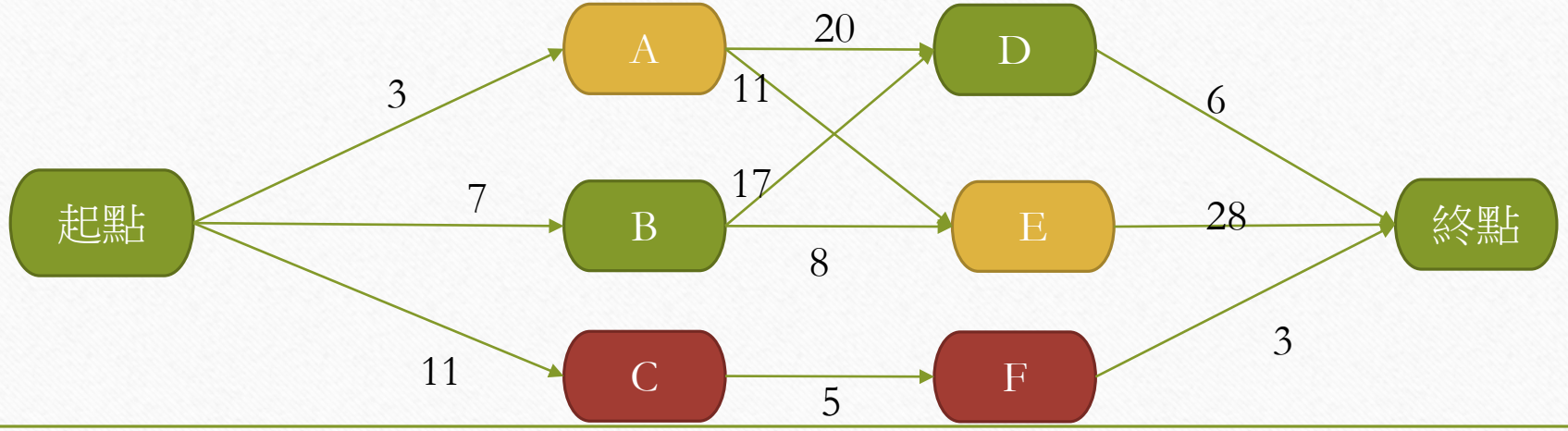
- 暴力法
- 貪婪法
- 分治法
- 回溯法
- 分支限界法
- 動態規劃

暴力法

- 最常用的演算法(?)
- 硬A，有多少算多少。
- 把所有可以計算的項目都計算出來。
- 相關：氣泡排序法

貪婪法

- 選擇目前最好的答案。
- 也就是局部最佳解。
- 貪婪法採用的是局部最佳，但不一定是最佳解。



分治法

- 把問題拆分成小部分，把小部分問題個別處理完成。
- 用來處理步驟很多的問題，或是可以劃分區域的問題。
- 相關：快速排序法。
- 有時候也會將複雜的問題簡化，如：在以排序的資料中進行折半搜尋演算法。

回溯法

- 把問題分成階段性。
- 對問題進行下一階段的運算。
- 當問題在該選項下被判斷為不可能則回溯到上一階段。
- 當選項被判斷為可能則紀錄為其中一個解。
- 比起暴力法，可以忽略不可能的選項，不用全部都計算完。
- 相關：深度優先搜尋。-八皇后

分支限界法

- 相同的把問題分割成階段
- 將每階段的解進行運算，排除不可能的項目。
- 廣度優先演算法。

動態規劃

- 與分治法很像，都是把大問題切割成小問題。
- 但這個小問題不同的地方是這些小問題是相似的或是相同的。
- 用來解決需要重複解決的問題。
- 舉例來說：爬樓梯問題：如果小明一次可以爬一階或兩階樓梯，請問如果爬10階樓梯有幾種爬法。